6. Keylime Verifier 컴포넌트 핵심 기능 분석

6.1 Verifier 개요 및 역할

Keylime Verifier는 TPM 기반 원격 증명 시스템의 핵심 컴포넌트로, 등록된 에이전트들의 지속적인 무결성 검증을 담당합니다. Verifier는 다음과 같은 주요 역할을 수행합니다:

6.1.1 지속적 무결성 검증

- **실시간 모니터링**: 등록된 에이전트들의 TPM 상태를 주기적으로 검증
- **Quote 검증**: TPM에서 생성된 Quote를 통해 플랫폼 무결성 확인
- **IMA 측정 검증**: 파일 시스템 무결성 측정 결과 분석
- **정책 기반 검증**: 사전 정의된 TPM 정책 및 런타임 정책에 따른 검증

6.1.2 상태 관리 및 추적

- **에이전트 상태 추적**: 각 에이전트의 운영 상태 및 증명 이력 관리
- **증명 카운터**: 성공적인 증명 횟수 및 마지막 증명 시간 추적
- **부팅 시간 검증**: 에이전트 재부팅 감지 및 IMA 측정 리스트 초기화

6.2 Tornado 웹 프레임워크 기반 아키텍처

6.2.1 비동기 처리 구조 (`keylime/cloud_verifier_tornado.py`)

Keylime Verifier는 Python Tornado 웹 프레임워크를 기반으로 구축되어 높은 동시성과 비동기 처리를 지원합니다:

```
import tornado.httpserver

import tornado.ioloop

import tornado.netutil

import tornado.process

import tornado.web

from concurrent.futures import ThreadPoolExecutor
```

6.2.2 주요 핸들러 클래스들

BaseHandler 클래스

```
class BaseHandler(tornado.web.RequestHandler):

def  prepare(self) -> None:

super().prepare()

def  write_error(self, status_code: int, **kwargs: Any) -> None:

self.set_header("Content-Type", "text/json")

# JSON 형태의 에러 응답 생성
```

AgentsHandler 클래스

- **GET 요청**: 에이전트 상태 조회 및 목록 반환
- **POST 요청**: 새로운 에이전트 등록
- **DELETE 요청**: 에이전트 제거 및 정리
- **PUT 요청**: 에이전트 정보 업데이트

VersionHandler 클래스

- API 버전 정보 제공
- 클라이언트-서버 간 버전 호환성 검증

6.2.3 비동기 처리 메커니즘

```
@contextmanager
def  session_context() -> Iterator[Session]:
"""데이터베이스 세션 컨텍스트 매니저"""
session_manager = SessionManager()
with session_manager.session_context(engine) as session:
yield session
```

6.3 데이터베이스 연동 및 세션 관리

6.3.1 SessionManager 클래스 (`keylime/db/keylime_db.py`)

```
class SessionManager:
engine: Optional[Engine]
_scoped_session: Optional[scoped_session]
def  __init__(self) -> None:
self.engine = None
self._scoped_session = None
def  make_session(self, engine: Engine) -> Session:
```

```
"""데이터베이스 세션 생성"""
```

6.3.2 데이터베이스 엔진 생성

```
def  make_engine(service: str, **engine_args: Any) -> Engine:
"""Keylime 서비스용 데이터베이스 엔진 생성"""
if service == "cloud_verifier":
config_service = "verifier"
url = config.get(config_service, "database_url")
# SQLite 또는 다른 데이터베이스 지원
```

6.3.3 데이터 모델 구조 (`keylime/db/verifier_db.py`)

VerfierMain 테이블

```
class VerfierMain(Base):
__tablename__ = "verifiermain"
agent_id = Column(String(80), primary_key=True)
operational_state = Column(Integer)
tpm_policy = Column(JSONPickleType(pickler=JSONPickler))
ima_policy = relationship("VerifierAllowlist")
mb_policy = relationship("VerifierMbpolicy")
attestation_count = Column(Integer)
last_received_quote = Column(Integer)
```

```
last_successful_attestation = Column(Integer)

tpm_clockinfo = Column(JSONPickleType(pickler=JSONPickler))
```

VerifierAllowlist 테이블

```
class VerifierAllowlist(Base):

__tablename__ = "allowlists"

name = Column(String(255), nullable=False)

checksum = Column(String(128))

generator = Column(Integer)

ima_policy = Column(Text().with_variant(Text(429400000), "mysql"))
```

6.4 공통 기능 모듈 (keylime/cloud_verifier_common.py)

6.4.1 Quote 응답 처리

```
def  process_quote_response(

agent: Dict[str, Any],

mb_policy: Optional[str],

runtime_policy: RuntimePolicyType,

json_response: Dict[str, Any],

agentAttestState: AgentAttestState,

) -> Failure:
```

```
"""Cloud Agent로부터의 응답 검증"""
```

이 함수는 다음과 같은 검증을 수행합니다:

- **공개키 검증**: 에이전트가 제공한 공개키 유효성 확인
- **해시 알고리즘 검증**: 허용된 TPM 해시 알고리즘 사용 여부 확인
- **암호화 알고리즘 검증**: 허용된 암호화 및 서명 알고리즘 확인
- **IMA 측정 리스트 검증**: 파일 무결성 측정 결과 분석
- **부팅 시간 검증**: 에이전트 재부팅 감지

6.4.2 V 키 준비 및 암호화

```
def  prepare_v(agent: Dict[str, Any]) -> Dict[str, bytes]:
"""V 키를 에이전트 공개키로 암호화"""
if agent.get("b64_encrypted_V", "") != "":
b64_encrypted_V = agent["b64_encrypted_V"]
logger.debug("Re-using cached encrypted V")
else:
# V를 공개키로 암호화
b64_encrypted_V = base64.b64encode(
crypto.rsa_encrypt(
crypto.rsa_import_pubkey(agent["public_key"]),
base64.b64decode(agent["v"])
)
)
```

6.4.3 Quote 요청 준비

```
def  prepare_get_quote(agent: Dict[str, Any]) -> Dict[str, Union[str, int]]:
"""Cloud Agent에 대한 Quote 요청 준비"""
agentAttestState = get_AgentAttestStates().get_by_agent_id(agent["agent_id"])
agent["nonce"] = tpm_util.random_password(20)
tmp_policy = ast.literal_eval(agent["tpm_policy"])
params = {
"nonce": agent["nonce"],
"mask": tpm_policy["mask"],
"ima_ml_entry": agentAttestState.get_next_ima_ml_entry(),
}
```

6.5 에이전트 상태 관리 (`keylime/agentstates.py`)

6.5.1 TPMClockInfo 데이터 클래스

```
@dataclass
class TPMClockInfo:
clock: int
resetcount: int
restartcount: int
safe: int
```

```
@classmethod
def from_dict(cls, dclki: Dict[str, int]) -> "TPMClockInfo":
return cls(
clock=dclki.get("clock", 0),
resetcount=dclki.get("resetCount", 0),
restartcount=dclki.get("restartCount", 0),
safe=dclki.get("safe", 1),
)
```

6.5.2 AgentAttestState 클래스

```
class AgentAttestState:
"""증분 증명을 지원하는 에이전트 증명 상태"""
agent_id: str
next_ima_ml_entry: int
boottime: int
tpm_clockinfo: TPMClockInfo
tpm_state: TPMState
ima_pcrs: Set[int]
ima_keyring: ImaKeyrings
```

6.6 정책 캐시 및 성능 최적화

6.6.1 글로벌 정책 캐시

```
GLOBAL_POLICY_CACHE: Dict[str, Dict[str, str]] = {}
```

```
def  verifier_read_policy_from_cache(ima_policy_data: Dict[str, str]) -> str:
"""IMA 정책을 캐시에서 읽기"""
checksum = ima_policy_data.get("checksum", "")
agent_id = ima_policy_data.get("agent_id", "")
if agent_id not in GLOBAL_POLICY_CACHE:
GLOBAL_POLICY_CACHE[agent_id] = {}
```

6.6.2 증명 상태 저장

```
def  store_attestation_state(agentAttestState: AgentAttestState) -> None:
"""IMA 로그가 평가된 경우에만 저장"""
if agentAttestState.get_ima_pcrs():
with session_context() as session:
update_agent = session.query(VerfierMain).get(agentAttestState.get_agent_id())
update_agent.boottime = agentAttestState.get_boottime()
update_agent.next_ima_ml_entry = agentAttestState.get_next_ima_ml_entry()
```

6.7 오류 처리 및 복구 메커니즘

6.7.1 실패 관리

```
def  prepare_error(agent: Dict[str, Any], msgtype: str = "revocation", event: Op
"""검증 문제 발생 시 철회 메시지 준비"""
revocation = {
"type": msgtype,
"ip": agent["ip"],
"agent_id": agent["agent_id"],
"event_time": time.asctime(),
}
```

6.7.2 데이터베이스 트랜잭션 관리

```
try:
with session_context() as session:
# 데이터베이스 작업 수행
session.add(update_agent)
# session.commit()은 컨텍스트 매니저에 의해 자동 호출
except SQLAlchemyError as e:
logger.error("SQLAlchemy Error: %s", e)
```

6.8 API 엔드포인트 및 RESTful 인터페이스

6.8.1 에이전트 관리 API

- **GET /agents/{agent_id}**: 특정 에이전트 상태 조회
- **GET /agents/**: 모든 에이전트 목록 조회
- **POST /agents/{agent_id}**: 새 에이전트 등록
- **DELETE /agents/{agent_id}**: 에이전트 제거

6.8.2 입력 검증 및 보안

```
def __validate_input(self, method: str) -> Tuple[Optional[Dict[str, Union[str,
"""API 입력 검증"""
if not validators.valid_agent_id(agent_id):
web_util.echo_json_response(self, 400, "agent_id not valid")
return None, None
```

6.9 멀티프로세싱 및 동시성 처리

6.9.1 프로세스 시작 방법 설정

```
try:
multiprocessing.set_start_method("fork")
except RuntimeError:
# set_start_method가 여러 번 호출될 때 발생할 수 있음
pass
```

6.9.2 ThreadPoolExecutor 활용

```
from concurrent.futures import ThreadPoolExecutor

# 비동기 작업 처리를 위한 스레드 풀 사용
```

이러한 구조를 통해 Keylime Verifier는 높은 성능과 확장성을 제공하면서도 안정적인 TPM 기반 원격 증명 서비스를 구현합니다.